VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

$\sim \cdots * \cdots \sim$



Computer Engineering Project

Week 2 Progress Report

REAL-TIME PERSONAL PROTECTIVE EQUIPMENT (PPE) MONITORING SYSTEM USING EDGE AI AND IOTs

Instructor: **Dr. Vo Tuan Binh**

Name	ID
Pham Ngoc Mai Anh	2352063
Tran Quoc Hao	2352299
Nguyen Minh Khue	2352631

Ho Chi Minh City, 2026

Contents

1 Dataset Preparation and Pre-processing **3**
1.1 Class Definition and Target Characteristics . . . 3
1.2 Data Collection and Format Standardization . . . 3
1.2.1 Open-Source Aggregation . . . 3
1.2.2 Annotation Conversion Pipeline . . . 4
1.3 Advanced Data Augmentation Strategies . . . 4

2 Analysis of Loss Functions: CIoU, DFL, and Class Loss **5**
2.1 Bounding Box Regression (Localization Loss): The Evolution to CIoU . . . 5
2.2 Distribution Focal Loss (DFL) . . . 6
2.3 Classification Loss: Multi-label Binary Cross-Entropy (BCE) . . . 6

3 Architectural Evolution: Application in YOLOv26 **8**
3.1 Re-evaluating Loss Functions in YOLOv26 . . . 8
3.2 Key Improvements and Overcoming Legacy Bottlenecks . . . 8

4 Model Optimization and Deployment **10**
4.1 Machine Learning Frameworks . . . 10
4.1.1 TensorFlow . . . 10
4.1.2 PyTorch . . . 10
4.1.3 Framework Comparison and Selection . . . 11
4.2 Edge Acceleration Tools . . . 12
4.2.1 TensorFlow Lite (TFLite) . . . 12
4.2.2 NVIDIA TensorRT . . . 12
4.2.3 ONNX and ONNX Runtime . . . 13
4.2.4 Tool Comparison . . . 13

5 System Architecture and IoT Telemetry Design **14**
5.1 The Bottleneck of Synchronous Execution . . . 14
5.2 Asynchronous Multi-threading Pipeline . . . 14

5.3 MQTT Protocol Integration and QoS Evaluation 15
5.3.1 Quality of Service (QoS) Selection 15
5.3.2 Extended JSON Payload Schema 15

1 Dataset Preparation and Pre-processing

1.1 Class Definition and Target Characteristics

The operational reliability of any deep learning model is strictly bounded by the statistical distribution of its training data. In the context of industrial surveillance, the system is strictly tailored to detect two primary safety components. Understanding the physical and visual characteristics of these two classes is critical for optimizing the Mean Average Precision (mAP) of the Edge AI system.

1. **Hard Hats (Class 0):** The fundamental physical defense against fatal head traumas caused by falling debris. The primary computer vision challenge with this class is the severe scale variation. In typical surveillance footage captured from elevated angles, workers in the background possess a minimal pixel footprint for their heads. According to the Intersection over Union (IoU) metric used in object detection:

$$IoU = \frac{Area(Box_{truth} \cap Box_{pred})}{Area(Box_{truth} \cup Box_{pred})} \tag{1}$$

   A minor bounding box deviation of just a few pixels when predicting a distant hard hat drastically reduces the IoU score. This mathematical sensitivity requires the dataset to be heavily populated with high-density, small-scale objects.

2. **Safety Vests (Class 1):** High-visibility gear essential for preventing deadly heavy machinery collisions. Unlike rigid hard hats, vests are highly deformable objects. The visual representation changes drastically based on the worker's posture (bending, walking), folding of the fabric, and reflective glare under direct sunlight. Therefore, the dataset must account for severe morphological changes and intra-class occlusions.

1.2 Data Collection and Format Standardization

Curating a robust dataset for Vietnamese construction site environments requires a hybrid data collection approach, as generic open-source datasets often lack the specific environmental contexts required for this project.

1.2.1 Open-Source Aggregation

We aggregated curated subsets from established repositories, primarily Roboflow Universe [1]. The data selection prioritized images with steep oblique camera angles and varying illumination levels (morning, noon, and overcast conditions) to prevent the model from overfitting to a specific lighting environment or viewing perspective.

1.2.2 Annotation Conversion Pipeline

Raw datasets frequently utilize the COCO JSON format, which represents bounding boxes as $[x_{min}, y_{min}, width, height]$ in absolute pixel values. However, the YOLO architecture strictly requires annotations in a normalized `.txt` format. We engineered a Python preprocessing script to convert all annotations using the following normalization formulas:

$$x_{center} = \frac{x_{min} + (width/2)}{Image_{width}}, \quad y_{center} = \frac{y_{min} + (height/2)}{Image_{height}} \tag{2}$$

$$w_{norm} = \frac{width}{Image_{width}}, \quad h_{norm} = \frac{height}{Image_{height}} \tag{3}$$

The resulting YOLO TXT format ensures that bounding box coordinates are scale-invariant (ranging from 0.0 to 1.0), which is a mandatory prerequisite for YOLOv26 training.

1.3 Advanced Data Augmentation Strategies

To simulate the harsh and dynamic visual conditions of active construction sites, we applied aggressive data augmentation techniques using the `Albumentations` library [2]. This pipeline expands the dataset mathematically without requiring manual labeling of new images.

- **Mosaic Augmentation:** This technique randomly crops and merges four different training images into a single standard grid. This forces the neural network to identify objects at a much smaller scale and within complex, unexpected contexts, directly solving the IoU degradation issue for distant hard hats.
- **Photometric Distortion (HSV Jittering):** Construction gear comes in various colors. To prevent the model from mapping "Hard Hat" strictly to the color yellow, we applied random shifts in Hue (± 0.015), Saturation (± 0.7), and Value (± 0.4). This forces the model to learn the structural shape of the helmet rather than its color.
- **Random Cutout / Coarse Dropout:** We synthetically masked rectangular regions (ranging from 10×10 to 20×20 pixels) across 10% of the images. This dropout forces the network to rely on partial features (e.g., detecting a worker based only on the shoulder of the vest when the torso is blocked by scaffolding), maximizing robustness against severe physical occlusions.

2 Analysis of Loss Functions: CIoU, DFL, and Class Loss

2.1 Bounding Box Regression (Localization Loss): The Evolution to CIoU

The localization loss is responsible for penalizing the spatial misalignment between the predicted bounding box B and the ground-truth box B^{gt}. Over the years, this metric has evolved significantly to address the shortcomings of its predecessors.

1. Traditional IoU to GIoU: The original Intersection over Union (IoU) simply measures the overlap area. However, if B and B^{gt} do not intersect, $IoU = 0$, leading to a gradient vanishing problem where the network stops learning. Generalized IoU (GIoU) [3] addressed this by introducing the smallest enclosing convex hull C. As shown by Rezatofighi et al. [3], GIoU is a proper metric (non-negative, symmetric, satisfying the triangle inequality) that remains scale-invariant while always being a lower bound of IoU. While GIoU provides a gradient for non-overlapping boxes, it suffers from slow convergence and tends to falsely expand the predicted box during early training phases.

2. DIoU (Distance IoU): Distance IoU (DIoU) significantly accelerates convergence by directly penalizing the distance between the center points of the two boxes. However, DIoU only considers the overlap area and central point distance, completely ignoring the structural shape (aspect ratio) of the bounding box.

3. CIoU (Complete Intersection over Union): To achieve optimal localization, the CIoU loss was introduced. It simultaneously optimizes three geometric factors: overlap area, central point distance, and aspect ratio consistency. The CIoU loss is formulated as:

$$L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \tag{4}$$

The three geometric factors are mathematically defined as follows:

- **Overlap Area ($1 - IoU$):** Ensures maximum intersection between the predicted and actual bounding boxes.
- **Central Point Distance ($\frac{\rho^2}{c^2}$):** $\rho^2(b, b^{gt})$ is the squared Euclidean distance between the center points of B and B^{gt}. Dividing by c^2 (the squared diagonal length of the convex hull C) normalizes this distance, ensuring the loss remains scale-invariant.
- **Aspect Ratio Consistency (αv):** The parameter v measures the consistency of the aspect ratio between the predicted dimensions (w, h) and the ground-truth dimensions (w^{gt}, h^{gt}):

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2 \tag{5}$$

The trade-off parameter α dynamically balances the importance of the aspect ratio:

$$\alpha = \frac{v}{(1 - IoU) + v} \tag{6}$$

When the overlap is low, α decreases, forcing the model to prioritize minimizing the distance between centers. As the overlap increases, α grows, directing the network to finely tune the structural shape.

2.2 Distribution Focal Loss (DFL)

Standard bounding box regression methods (such as CIoU) force the network to predict rigid, deterministic coordinates, effectively modeling the output as a Dirac delta distribution. However, in practical applications, the edges of objects—such as a small hardhat at a distance, or a safety vest partially occluded by scaffolding—are often blurry and ambiguous. Forcing the network to pinpoint a single absolute pixel under such uncertainty leads to suboptimal learning.

Distribution Focal Loss (DFL), originally proposed by Li et al. [5] as part of the Generalized Focal Loss (GFL) framework, introduces an advanced probabilistic paradigm: it models the continuous coordinates of the bounding box edges as a discrete probability distribution (referred to as the "General distribution" in [5]), rather than assuming a rigid Dirac delta or an overly simplistic Gaussian distribution. Instead of outputting a single coordinate y, the network predicts a probability distribution over a grid of discrete locations.

Suppose the continuous ground-truth target coordinate y falls between two neighboring discrete grid points y_i and y_{i+1} (such that $y_i \leq y \leq y_{i+1}$). The network uses a Softmax layer to predict the probabilities S_i and S_{i+1} for these two nearest points. DFL optimizes the model by forcing it to rapidly focus its probability mass on the values immediately adjacent to the true label:

$$L_{DFL}(S_i, S_{i+1}) = -\left((y_{i+1} - y)\log(S_i) + (y - y_i)\log(S_{i+1})\right) \tag{7}$$

By minimizing this cross-entropy-based metric, DFL compels the network to output a highly concentrated (focal) probability distribution around the actual boundary. This probabilistic approach grants the model the flexibility to express uncertainty when dealing with blurry edges or small, distant safety equipment, thereby significantly enhancing the robustness of spatial localization in complex industrial environments.

2.3 Classification Loss: Multi-label Binary Cross-Entropy (BCE)

Modern YOLO architectures (such as YOLOv8 and YOLO11) utilize a Decoupled Head, an idea first introduced into the YOLO family by YOLOX [4], which strictly separates the spatial regression branch from the semantic classification branch. Ge et al. [4] showed that this decoupling both accelerates convergence and is essential for enabling end-to-end (NMS-free) YOLO variants, since a coupled head suffers a much larger performance drop when the

one-to-one matching required for end-to-end detection is applied. This decoupling resolves the inherent conflict between tasks: localization requires features that are highly sensitive to spatial coordinates, whereas classification requires spatial invariance.

In traditional detection architectures, the classification head often employed the Softmax function, which assumes that classes are mutually exclusive (i.e., the probabilities across all classes sum to 1). However, in real-world applications—particularly in occupational safety datasets—a single bounding box frequently contains overlapping semantics. For example, a bounding box targeting a *hardhat* or a *safety-vest* will inevitably encompass the facial or bodily features of a *person*. If Softmax is applied, increasing the model's confidence for the *hardhat* class will artificially suppress the confidence for the *person* class, leading to conflicting gradients and suboptimal learning.

To resolve this gradient suppression, the decoupled classification head treats the detection of each category as a completely independent binary classification problem. It first applies the Sigmoid activation function to the raw logit z_i of each class independently:

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \tag{8}$$

The model then evaluates the classification error using the multi-label Binary Cross-Entropy (BCE) loss. For a given set of predictions over C independent classes (e.g., *person, hardhat, safety-vest*), the loss is formulated as:

$$L_{cls} = \frac{1}{N} \sum_{j=1}^{N} \sum_{i=1}^{C} - \left[y_{j,i} \log(\hat{y}_{j,i}) + (1 - y_{j,i}) \log(1 - \hat{y}_{j,i}) \right] \tag{9}$$

Where:

- N is the total number of predicted bounding boxes.
- C is the total number of classes.
- $y_{j,i} \in \{0, 1\}$ represents the ground-truth label indicating whether box j belongs to class i.
- $\hat{y}_{j,i} \in (0, 1)$ is the predicted probability that box j contains class i.

By utilizing the BCE loss, the network optimizes the probability of each class independently. This allows the model to confidently output high probabilities for multiple overlapping classes simultaneously, ensuring that learning the features of a *hardhat* or *safety-vest* does not inadvertently penalize the network's ability to recognize the *person* wearing them.

3 Architectural Evolution: Application in YOLOv26

While the mathematical formulations of CIoU, BCE, and DFL provided significant accuracy boosts in architectures like YOLOv8 and YOLO11, the deployment of these models on resource-constrained Edge AI devices revealed critical operational bottlenecks. YOLOv26 introduces a paradigm shift: moving away from mathematical over-complication in the detection head toward a streamlined, End-to-End architecture optimized for real-time inference.

3.1 Re-evaluating Loss Functions in YOLOv26

YOLOv26 fundamentally alters how the aforementioned loss functions are applied to balance precision with computational efficiency:

- **Retention of CIoU and BCE:** YOLOv26 maintains the Decoupled Head architecture, continuing to leverage CIoU for robust bounding box regression and multi-label BCE for independent semantic classification. These functions remain foundational because they resolve the core spatial and semantic challenges (aspect ratio distortion and overlapping semantics) without adding computational overhead during the inference phase.
- **The Removal of Distribution Focal Loss (DFL):** Although DFL is mathematically elegant for handling ambiguous boundaries, YOLOv26 deliberately **removes** DFL from its architecture. DFL requires the model to output a continuous probability distribution across a discrete grid, which significantly increases the memory footprint and the number of tensor operations in the regression head. By reverting to direct coordinate regression, YOLOv26 sacrifices a marginal amount of edge-case boundary precision to achieve a massive leap in inference speed and hardware compatibility (e.g., for deployment on Jetson Nano or FPGAs).

3.2 Key Improvements and Overcoming Legacy Bottlenecks

YOLOv26 overcomes several historical disadvantages of the YOLO family through its revamped Detection Head:

1. End-to-End NMS-Free Detection (One-to-One Head): Historically, YOLO models predicted thousands of redundant bounding boxes for a single object. To clean up these duplicates, models relied on Non-Maximum Suppression (NMS) as a post-processing step. However, NMS introduces two severe disadvantages:

- *Latency Bottleneck:* NMS is a sequential, CPU-bound operation that creates a severe bottleneck, nullifying the speed advantages of GPU acceleration.

- *Occlusion Suppression:* In densely packed environments (e.g., workers standing shoulder-to-shoulder), NMS often incorrectly suppresses valid bounding boxes, mistaking them for duplicates of the same object.

YOLOv26 overcomes this by introducing a **Dual-Head** architecture featuring a primary **One-to-One prediction head**. The network is trained using advanced bipartite matching algorithms to natively output the single best bounding box per object. This entirely eliminates the need for NMS, drastically reducing latency and improving recall in dense crowds.

This design directly builds on the *consistent dual assignments* strategy proposed by YOLOv10 [6]. In that work, Wang et al. [6] attach an auxiliary one-to-many head alongside the one-to-one head during training: the one-to-many branch supplies rich supervisory signal to the backbone and neck (as in conventional YOLO training), while a *consistent matching metric* ($\alpha_{o2o} = r \cdot \alpha_{o2m}$, $\beta_{o2o} = r \cdot \beta_{o2m}$) harmonizes the two branches so that the best positive sample under one-to-many assignment is also the best sample selected by the one-to-one branch. At inference time, only the lightweight one-to-one head is retained, so NMS-free, end-to-end detection is achieved without incurring extra inference cost. This approach improves on earlier one-to-one matching attempts for CNN-based detectors, such as the optional end-to-end variant explored in YOLOX [4], which added one-to-one label assignment and stop-gradient on top of a decoupled head but observed a moderate accuracy drop (about 0.8 AP) relative to the standard one-to-many-trained model, since it lacked such a consistency mechanism between assignments.

2. Edge AI Optimization and Deployment Friction: Past iterations struggled with deployment friction; exporting models with complex DFL layers and NMS algorithms to frameworks like TensorRT, ONNX, or OpenVINO often caused operator unsupported errors. By stripping away DFL and NMS, YOLOv26 offers a highly streamlined, pure neural-network pipeline. This results in faster compilation times, lower memory consumption, and a significantly higher Frames Per Second (FPS) rate on embedded systems, making it the definitive architecture for real-time industrial safety monitoring.

4 Model Optimization and Deployment

4.1 Machine Learning Frameworks

4.1.1 TensorFlow

TensorFlow is an open-source platform for machine learning using data flow graphs. Nodes in the graph represent mathematical operations, while the graph edges represent the multi-dimensional data arrays (tensors) that flow between them. This flexible architecture allows machine learning algorithms to be described as a graph of connected operations. They can be trained and executed on GPUs, CPUs, and TPUs across various platforms without rewriting code, ranging from portable devices to desktops to high-end servers.

There are three distinct parts that define the TensorFlow workflow, namely preprocessing of data, building the model, and training the model to make predictions. The framework inputs data as a multidimensional array called tensors and executes in two different fashions. The primary method is by building a computational graph that defines a dataflow for training the model. The second, and often more intuitive method, is using eager execution, which follows imperative programming principles and evaluates operations immediately.

TensorFlow can be used to develop models for various tasks, including natural language processing, image recognition, handwriting recognition, and different computational-based simulations such as partial differential equations. The key benefits of TensorFlow are in its ability to execute low-level operations across many acceleration platforms, automatic computation of gradients, production-level scalability, and interoperable graph exportation. By providing Keras as a high-level API and eager execution as an alternative to the dataflow paradigm on TensorFlow, it's always easy to write code comfortably. As the original developer of TensorFlow, Google still strongly backs the library and has catalyzed the rapid pace of its development. For example Google has created an online hub for sharing the many different models created by users.

4.1.2 PyTorch

PyTorch is an open-source machine learning framework that is known for its flexibility, ease of use, and performance in modern AI applications. This is enabled in part by its compatibility with the popular Python high-level programming language favored by machine learning developers and AI researchers.

PyTorch supports the two main AI workloads: training and inference. During training, PyTorch aids in teaching a neural network by orchestrating a continuous cycle where input data, stored as Tensors, flows through the model to generate predictions. The framework's defining feature is its automatic differentiation engine, autograd, which automatically calculates exactly how the model's parameters need to change to reduce errors (backpropagation).

- Flexible and Fast: PyTorch is built on an intuitive Python frontend that focuses on readability and rapid iteration. However, modern PyTorch is also built for speed. With features like TorchScript and TorchDynamo, developers can seamlessly transition from eager experimentation to high-performance production deployment.
- The Generative AI Standard: PyTorch is the native language of the generative AI revolution. It is the framework of choice for building large language models (LLMs) and diffusion models due to its support for distributed training across thousands of GPUs using FSDP (Fully Sharded Data Parallel).
- Developer Ecosystem: The PyTorch API has remained consistent and user-friendly, making it accessible for beginners while offering deep control for experts. Its massive ecosystem includes libraries for computer vision, natural language processing (NLP), and reinforcement learning, ensuring that if a new AI technique exists, there is likely already a PyTorch implementation for it.

4.1.3 Framework Comparison and Selection

Criteria	**TensorFlow**	**PyTorch**
Community & Ecosystem	Strong presence in industry and production environments. Extensive documentation, but slightly fragmented due to the historical transition from TF1 to TF2.	Dominates AI research and academia. Backed by massive open-source ecosystems (e.g., Hugging Face, Torchvision) and state-of-the-art model repositories.
Customizability & Flexibility	Uses static computational graphs (though eager execution is available). Writing custom layers or debugging deep architectural changes can sometimes be rigid.	Pythonic and relies entirely on dynamic computational graphs (autograd). Highly intuitive for rapid prototyping, modifying architectures, and line-by-line debugging.
Deployment Seamlessness	Highly mature, end-to-end ecosystem. Tools like TensorFlow Serving and TFLite make the transition from training to edge production very streamlined.	Historically required more manual effort, but currently well-supported via TorchServe and seamless exportation to intermediate formats like ONNX for hardware-specific deployment.

4.2 Edge Acceleration Tools

4.2.1 TensorFlow Lite (TFLite)

TensorFlow Lite is an open-source deep learning framework designed for on-device inference, commonly referred to as Edge Computing. It enables developers to deploy trained machine learning models directly on edge devices such as mobile phones, embedded systems and IoT devices without relying on cloud resources.

- Train the Model: Build and train the machine learning model using TensorFlow on high-performance systems.
- Convert to TensorFlow Lite: Convert the trained model into .tflite format using the TensorFlow Lite Converter, applying optimizations if required.
- Optimize the Model: Apply techniques like quantization to reduce model size and improve inference speed.
- Deploy on Edge Device: Integrate the .tflite model into mobile, embedded or IoT applications.
- Run Inference: Execute the model using the TensorFlow Lite interpreter for fast, on-device predictions.

4.2.2 NVIDIA TensorRT

NVIDIA TensorRTis an SDK for optimizing deep learning inference on NVIDIA GPUs. It takes trained models from frameworks such as PyTorch and ONNX, and compiles them into engines for GPU deployment with support for mixed precision (FP32/FP16/BF16/FP8/INT8/FP4/INT4), dynamic shapes, and specialized optimizations for transformers and large language models (LLMs).

TensorRT includes libraries that optimize neural network models trained on all major frameworks, calibrate them for lower precision with high accuracy, and deploy them to hyperscale data centers, workstations, laptops, and edge devices. TensorRT optimizes inference using quantization, layer and tensor fusion, and kernel tuning techniques.

NVIDIA TensorRT Model Optimizer provides easy-to-use quantization techniques, including post-training quantization and quantization-aware training to compress your models. FP8, FP4, INT8, INT4, and advanced techniques such as AWQ are supported for your deep learning inference optimization needs. Quantized inference significantly minimizes latency and memory bandwidth, which is required for many real-time services, autonomous and embedded applications.

4.2.3 ONNX and ONNX Runtime

ONNX Runtime is a cross-platform machine-learning model accelerator, with a flexible interface to integrate hardware-specific libraries. ONNX Runtime can be used with models from PyTorch, Tensorflow/Keras, TFLite, scikit-learn, and other frameworks.

At its core, ONNX defines a common format for representing deep learning models, referred to as the ONNX model format. This format consists of a computational graph, where nodes represent operations, and edges represent the flow of data. Each node in the graph corresponds to a specific operation, such as convolution, pooling, or activation, and includes attributes that define its behavior.

4.2.4 Tool Comparison

Criteria	**TensorFlow Lite (TFLite)**	**NVIDIA TensorRT**	**ONNX Runtime**
Supported Platforms	CPUs, Mobile (Android/iOS), Microcontrollers, and Edge TPUs.	NVIDIA GPUs exclusively (Datacenter GPUs and Jetson Edge devices).	Cross-platform (CPUs, GPUs, DSPs, NPUs, Web browsers).
Inference Speed	Highly optimized for ARM processors and low-power IoT devices. Moderate performance on GPUs.	Achieves the absolute lowest inference latency and highest throughput on NVIDIA hardware.	Fast and versatile baseline; can leverage different execution providers (including TensorRT) but adds slight architectural overhead.
Key Optimizations	FlatBuffer conversion, INT8 quantization, and minimal memory footprint.	Layer fusion, kernel auto-tuning, optimized CUDA execution, and FP16/INT8 calibration.	Graph optimizations (node fusion, constant folding), and a pluggable execution engine.

5 System Architecture and IoT Telemetry Design

5.1 The Bottleneck of Synchronous Execution

Deploying deep learning models on embedded edge platforms (e.g., Raspberry Pi or Jetson Nano) introduces severe computer engineering constraints. A naive software implementation executes tasks synchronously in a single `while True` loop: capturing a frame, running neural inference, and sending network alerts sequentially.

This synchronous design is fundamentally flawed for real-time edge applications. Neural inference dictates the loop's speed. If the AI takes 100ms to process a frame, the camera buffer is blocked for 100ms. Furthermore, if a network timeout occurs while sending an alert (e.g., taking 2 seconds), the entire video stream freezes, causing catastrophic frame-dropping and buffer overflows.

5.2 Asynchronous Multi-threading Pipeline

To decouple the hardware I/O limits from the neural computation limits, we architected an **Asynchronous Multi-threading Pipeline** based on the classic Producer-Consumer software design pattern [7]. The system utilizes Python's thread-safe `queue.Queue` to pass data between three independent, concurrently running threads:

1. **Data Acquisition Thread (Camera I/O):** Acting as the primary Producer, this thread continuously reads frames from the RTSP/USB camera stream using OpenCV. It pushes frames into an *Inference Queue*. To prevent memory overflow if the AI falls behind, the queue size is strictly limited to 1 (LIFO - Last In, First Out). If a new frame arrives before the AI retrieves the previous one, the old frame is instantly overwritten. This guarantees the AI always processes real-time data with zero accumulated latency.

2. **Edge Processing Thread (AI Inference):** This is the core computation thread. It acts as a Consumer for the Inference Queue and a Producer for the network. It continuously pops the latest frame, executes the quantized YOLO inference, and evaluates the bounding boxes. If a safety violation is detected, it does not execute blocking network calls. Instead, it pushes the violation data into a separate *Telemetry Queue*.

3. **IoT Telemetry Thread (MQTT Publisher):** This dedicated networking thread monitors the Telemetry Queue. When a violation event is logged, it instantly extracts the data and handles the TCP/IP network transmission in the background. If the site's network disconnects, this thread handles connection retries independently, ensuring the Camera and AI threads continue running at maximum FPS without interruption.

5.3 MQTT Protocol Integration and QoS Evaluation

For the IoT communication layer, HTTP REST APIs were rejected due to their heavy header overhead and synchronous Request/Response model. Instead, we implemented **MQTT (Message Queuing Telemetry Transport)** using the **Eclipse Mosquitto** broker [8]. MQTT is a lightweight, publish-subscribe protocol explicitly designed for low-bandwidth, high-latency IoT networks.

5.3.1 Quality of Service (QoS) Selection

MQTT guarantees message delivery through three QoS levels. We evaluated these levels to determine the optimal balance between system latency and critical safety reliability:

QoS Level	Mechanism	Project Suitability
QoS 0	At most once (Fire and forget). No acknowledgment.	Rejected. Unreliable over unstable 4G construction networks. Critical alerts could be lost.
QoS 1	At least once. Requires acknowledgment (PUBACK).	**Selected.** Guarantees delivery. Minimal latency overhead. Duplicate alerts are easily filtered by the dashboard.
QoS 2	Exactly once. Four-step handshake process.	Rejected. The heavy handshake process increases latency and bandwidth consumption unnecessarily.

Table 1: Evaluation of MQTT Quality of Service levels for Edge Telemetry.

By configuring the Paho-MQTT client to utilize **QoS 1**, the system guarantees that no safety violation is ever lost due to a dropped cellular connection, while avoiding the heavy network overhead of QoS 2.

5.3.2 Extended JSON Payload Schema

To minimize bandwidth, the Edge device acts as a Publisher that transmits structured telemetry text rather than raw video feeds. When Thread 2 detects a violation, it generates a comprehensive JSON payload [9]. Beyond just the safety alert, we integrated edge hardware telemetry (CPU temperature and utilization) into the schema to monitor thermal throttling remotely:

```
{
  "device_id": "edge_node_jetson_01",
  "zone": "scaffolding_sector_B",
```

```
  "timestamp": "2026-09-24T08:35:12Z",
  "violation_data": {
      "type": "missing_hardhat",
      "confidence": 0.92,
      "bounding_box": [120, 45, 200, 215]
  },
  "hardware_telemetry": {
      "cpu_temp_c": 68.5,
      "ram_usage_percent": 45.2,
      "inference_fps": 22.4
  }
}
```

The centralized dashboard subscribes to the `site/+/alerts` topic (using the `+` wildcard to listen to all zones). Upon receiving this JSON payload, the dashboard updates the web interface, logs the event into a time-series database, and triggers physical on-site actuators (e.g., warning sirens) if necessary.

References

[1] Roboflow, "Construction Site Safety Dataset," [Online]. Available: https://universe.roboflow.com/, 2024.

[2] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and Flexible Image Augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.

[3] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression," *arXiv preprint arXiv:1902.09630*, 2019.

[4] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO Series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[5] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," *arXiv preprint arXiv:2006.04388*, 2020.

[6] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-Time End-to-End Object Detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. (*arXiv preprint arXiv:2405.14458*).

[7] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[8] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *The Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017.

[9] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, Dec. 2017. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8259